

# Smart E-Mobility Hub

## Project Report Guideline and Assessment Rubric

---

### Software Engineering — HK261

This document specifies the expected structure and contents of the final project report, the required analysis and design artifacts, testing documentation, and final presentation requirements.

## 1. Purpose and General Expectations

The project concerns the design and implementation of a Smart E-Mobility Hub system for coordinating electric mobility resources in the VNU-HCM Urban Area. The system may use real or simulated operational data. The main software engineering challenges include state management, resource allocation, reservation handling, charging scheduling, vehicle dispatching, event handling, and what-if simulation.

The final report should document the complete software engineering process followed by the group. The report is not intended to be a collection of screenshots or disconnected UML diagrams. Requirements, design decisions, implementation, and testing should be consistent with one another.

## 2. Requirements Analysis

This section answers the question: What must the system do? Requirements should be derived from the project problem and should provide a clear basis for the later design, implementation, and testing sections.

### 2.1 Stakeholders and Actors

Identify the important stakeholders and system actors. For each stakeholder or actor, explain the role, main goals, and relevant interactions with the system. Actors should be defined according to the chosen system boundary rather than copied mechanically from examples.

### 2.2 Functional Requirements

Specify the functional requirements of the system in a clear and identifiable form. The required system scope should normally address the following core capabilities:

- View Mobility Hubs and their current resource status.
- Find suitable rental electric vehicles and view availability and battery information.
- Reserve rental electric vehicles.
- Pick up and return rental electric vehicles.
- Reserve parking spaces.
- Request or schedule charging services.
- Support users of private electric vehicles in reserving parking and charging resources.
- Monitor vehicles, parking capacity, charging points, and Hub utilization.
- Handle operational incidents such as unavailable vehicles or failed charging points.
- Coordinate or redistribute rental vehicles among Mobility Hubs.
- Coordinate charging priorities and schedules.
- Run What-if Simulation scenarios and generate appropriate coordination recommendations.

## 2.3 Non-functional Requirements

Specify important non-functional requirements that are relevant to the proposed system. These may include performance, usability, reliability, availability, maintainability, security, scalability, data consistency, and other quality attributes. Requirements should be as concrete and verifiable as reasonably possible.

## 2.4 Business Rules and Constraints

State important rules that constrain system behavior and the use of shared resources. Examples include reservation conflicts, capacity limits, invalid state transitions, charging-point availability, and rules for handling failures. The submitted rules should reflect the group's actual design.

## 2.5 Use-case Modeling

Use-case diagrams are mandatory. Students should create as many use-case diagrams as necessary to represent the system clearly. A single excessively large diagram should be avoided. Use-case tables are encouraged to complement the diagrams.

# 3. System Design

This section answers the question: How is the system designed to satisfy the requirements? The design should be concrete enough that the relationship between requirements, software structure, behavior, and implementation can be understood.

## 3.1 Software Architecture

A concrete software architecture diagram is mandatory. The diagram should show the major software components or modules of the proposed system and the relationships or communication among them. Merely naming an architectural style (for example, MVC or layered architecture) is not sufficient.

The report should explain:

- the responsibilities of the major components or modules;
- how components communicate or depend on one another;
- where important business logic is located;
- how data is stored and accessed, where applicable;
- how the What-if Simulation subsystem interacts with the operational system state.

Architecture decisions must be justified. The report should explain why the chosen architecture is suitable for the project's requirements and discuss important benefits, limitations, or trade-offs. Where relevant, students may briefly compare the chosen approach with reasonable alternatives.

## 3.2 Structural and Data Design

### Class Diagram

If the system uses object-oriented design, one or more design-level class diagrams are required. The diagrams should represent the actual software design rather than only a conceptual list of domain entities.

Class diagrams should show, where relevant:

- classes and interfaces;
- important attributes;
- important operations;
- associations and multiplicities;

- inheritance and interface realization;
- aggregation or composition where meaningful.

### ERD / Data Model

If the system uses persistent structured data, an appropriate data model is required. For a relational database, this should normally be an Entity-Relationship Diagram (ERD). The ERD should show the major entities, relationships, primary/foreign keys as appropriate, and relevant cardinalities.

## 3.3 Behavioral and Interaction Design

Students must use appropriate UML behavioral and interaction diagrams to explain important non-trivial processes. The choice of diagram should depend on the aspect of the process being illustrated.

### Activity Diagrams

Use activity diagrams when the main concern is workflow, control flow, decision logic, concurrency, or alternative paths. Suitable examples include charging allocation, reservation processing, incident handling, and simulation workflows.

### Sequence Diagrams

Use sequence diagrams when the main concern is interaction among actors, components, services, or objects and the ordering of messages. Sequence diagrams should be consistent with the architecture and major software responsibilities.

### State Machine Diagrams

State machine diagrams should be included where state-dependent behavior is important. Examples may include rental vehicles, reservations, charging sessions, or other entities with meaningful lifecycle states and transitions. If no state machine diagram is provided, the omission should be justifiable based on the system design.

## 3.4 User-interface Design

The report should present the design of the important user interfaces using wireframes, mockups, prototypes, or selected designed screens. The purpose of this section is to explain the interface design and user interaction.

The UI design discussion should cover:

- major user flows and navigation;
- important information presented to each actor;
- important user actions and system feedback;
- design decisions related to usability and clarity;
- interfaces for both ordinary users and operators where applicable.

## 3.5 What-if Simulation and Coordination Design

What-if Simulation is a key capability of the project and must be described explicitly. The report should explain how hypothetical operating conditions are represented, simulated, evaluated, and converted into coordination recommendations.

At minimum, describe:

- scenario definition and configurable inputs;
- the operational state used as the simulation starting point;
- simulation logic and state evolution;

- impact measures or indicators;
- the logic used to generate coordination recommendations.

Typical scenarios may include:

- a sudden increase in demand at the Metro station or another Hub;
- a Hub reaching parking capacity;
- a surge in charging demand;
- partial charging-point failures;
- an excessive concentration or shortage of rental vehicles at one location.

## 4. Implementation

This section documents how the approved design was realized in software. It should focus on technically meaningful implementation choices rather than repeating the design section.

### 4.1 Technology Stack and Development Environment

Describe the main implementation technologies, such as programming languages, frameworks, database systems, libraries, external services or APIs, and deployment/runtime environment. Important technology choices should be briefly justified.

### 4.2 Implementation Structure

Explain how the software architecture and design map to the implementation. This may include major packages, modules, services, classes, persistence mechanisms, APIs, event-handling mechanisms, and the simulation subsystem. There is no need to document every class or source file.

### 4.3 Important Implementation Decisions

Discuss technically significant implementation issues. Examples include maintaining consistent resource state, resolving reservation conflicts, processing failures, synchronizing state updates, implementing charging allocation, or isolating simulated state from actual operational state.

### 4.4 Deviations and Implementation Limitations

Identify any important differences between the system design and the final implementation. Explain features that were simplified, changed, partially implemented, or omitted, together with the reason for each significant deviation.

## 5. Testing and Validation

Testing must be treated as a proper software engineering activity. The report should document both the planned testing process and the actual tests performed. A small collection of screenshots or ad hoc examples is not sufficient.

### 5.1 Test Plan

The test plan should normally include:

- testing objectives;
- scope of testing;
- features/functions to be tested;
- features not tested, if any, with justification;
- testing levels and types used;

- test environment and configuration;
- test data and data-preparation approach;
- roles and responsibilities;
- entry criteria;
- exit or completion criteria;
- test schedule or test organization;
- testing tools or automation tools, where applicable.

## 5.2 Test Design and Test Techniques

Explain how test cases were derived. Students should apply appropriate techniques taught in the course rather than creating arbitrary test inputs.

Depending on the tested functionality, techniques may include:

- requirements-based testing;
- use-case-based testing;
- equivalence partitioning;
- boundary-value analysis;
- decision-table testing;
- state-transition testing;
- unit or structural testing techniques;
- integration testing techniques.

## 5.3 Test Cases

Test cases should be documented in a consistent format and linked to the requirements or use cases they verify. The overall test set should include meaningful coverage of:

- normal operation;
- invalid inputs or operations;
- boundary and capacity conditions;
- resource conflicts;
- state transitions;
- equipment or resource failures;
- What-if Simulation scenarios;
- integration among important components.

## 5.4 Test Execution and Results

Summarize the actual execution of the test plan. Report the number of tests executed and their outcomes, discuss important failures, and explain how defects were addressed. Where appropriate, provide a concise defect summary including severity, related test case, status, and resolution.

## 6. Evaluation and Conclusion

The final section should evaluate the completed system critically and summarize the project.

It should discuss:

- the extent to which the original objectives and major requirements were achieved;
- important strengths of the system;
- known limitations and incomplete features;

- significant design or implementation trade-offs;
- lessons learned during the project;
- reasonable future improvements;
- a concise final conclusion.

### 6.1 Generative AI Usage Declaration

A transparent declaration of Generative AI use is mandatory. The declaration should identify the tools used, the purposes for which they were used, and the parts of the work that were generated, revised, checked, or substantially developed by the students. Students remain responsible for understanding and defending all submitted work.

## 7. Final Presentation and Demonstration

Each group will give a final presentation at the end of the semester. The total presentation time will be approximately 20 minutes, including approximately 3 minutes for the system demonstration.

The demonstration is mandatory and must be recorded in advance. Live demonstrations are not permitted. The recorded demonstration should show the important functionality of the completed system within the allocated time.

Additional requirements concerning presentation content, scheduling, submission procedure, and detailed assessment will be announced by the lecturer later.

## 8. Suggested Report Assessment Rubric

The following rubric may be used to assess the written report. The final presentation is assessed separately. The total grade of this assignment is divided evenly between the report and the presentation.

| Component                                  | Assessment focus                                                                                                                                                             | Weight |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Introduction                               | Problem context, objectives, scope, stakeholders, assumptions, constraints, and clarity of system boundary.                                                                  | 5%     |
| Requirements Analysis                      | Functional and non-functional requirements, business rules, stakeholder/actor analysis, multiple clear use-case diagrams where needed, and detailed use-case specifications. | 15%    |
| System Architecture                        | Concrete architecture, component responsibilities and interactions, consistency with requirements, and clear justification/trade-offs.                                       | 15%    |
| Structural and Data Design                 | Appropriate class diagram(s) when OOP is used, ERD/data model when applicable, correctness and consistency with the implementation.                                          | 10%    |
| Behavioral and Interaction Design          | Appropriate activity, sequence, and state-machine diagrams as applicable; sufficient coverage of important non-trivial behavior.                                             | 15%    |
| UI Design                                  | Clear design of important interfaces, navigation and user flows, usability rationale, and suitability for major actors.                                                      | 5%     |
| What-if Simulation and Coordination Design | Scenario definition, simulation logic, impact evaluation, and coordination recommendation mechanism.                                                                         | 10%    |
| Implementation                             | Technology choices, mapping from design to implementation, important implementation decisions, and transparent discussion of deviations/limitations.                         | 10%    |

|                                            |                                                                                                                                       |     |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-----|
| Testing and Validation                     | Quality and completeness of the test plan, test design techniques, test cases, execution evidence, result analysis, and traceability. | 10% |
| Evaluation, Conclusion, and AI Declaration | Critical evaluation, limitations, lessons learned, future work, conclusion, and transparent Generative AI usage declaration.          | 5%  |